Benchmarking LLM Architectures for Industrial Root Cause Analysis: Fine-Tuning vs. Retrieval-Augmentation

Shijun Feng
Ericsson AB
shijun.feng@ericsson.com

Department of Computer Science
XYZ Institute
jane@xyz.edu

***Abstract*—In modern industrial software ecosystems, the high volume of operational fault reports renders manual Root Cause Analysis (RCA) a significant maintenance bottleneck, often leading to delayed resolution and high operational costs. While Large Language Models (LLMs) show promise in automating diagnostic tasks, a systematic evaluation of architectural paradigms—specifically Supervised Fine-Tuning (SFT) versus Retrieval-Augmented Generation (RAG)—remains absent in specialized industrial domains. This study presents a rigorous benchmarking of three distinct architectures: an encoder-decoder model ($T5$), a lightweight decoder-only baseline ($GPT-2$), and a RAG framework, evaluated on a curated dataset of real-world industrial fault reports. Our findings reveal a critical architectural trade-off: $T5$ achieves superior lexical and structural fidelity (e.g., BLEU-4 of 0.1810 vs. GPT-2's 0.1210), whereas RAG provides the highest semantic alignment (*BERTScore* of 0.7715). These results suggest that while SFT is optimal for generating structured reports, RAG is superior for contextually grounded reasoning in complex fault scenarios. We propose a hybrid framework that leverages the structural precision of $T5$ and the semantic depth of RAG, providing a metric-driven roadmap for deploying intelligent RCA assistants in industrial software maintenance workflows.**

***Index Terms*—Root Cause Analysis (RCA), Large Language Models (LLM), T5, GPT-2, Retrieval-Augmented Generation (RAG), Software Maintenance, Fine-tuning.**

I. INTRODUCTION

Modern cloud-native networks generate a high volume of operational trouble reports, making Root Cause Analysis (RCA) a significant maintenance bottleneck. Currently, fault report management is largely manual and experience-driven, leading to delayed response times and redundant work—particularly in identifying duplicate issues across diverse environments [1]. The lack of automated frameworks increases the risk of inconsistent handling and human error in correlating system behaviors and logs [2].

While Large Language Models (LLMs) have shown potential in automating software engineering tasks [3], [4], existing studies often evaluate models in isolation. There is a lack of systematic comparison between architectural paradigms—specifically supervised fine-tuning (SFT) versus retrieval-augmentation—within specialized industrial domains. This study addresses this gap by benchmarking three distinct architectures: a fine-tuned encoder-decoder model ($T5$), a decoder-only autoregressive model ($GPT-2$), and a Retrieval-Augmented Generation (*RAG*) system using a curated dataset of textual fault reports.

The primary contributions of this paper are:

- **Performance Evaluation:** A quantitative analysis of Fine-Tuning $T5$ and $GPT-2$ models using lexical (BLEU, ROUGE) and semantic (BERTScore) metrics.
- **Architectural Comparison:** A side-by-side comparison of SFT versus RAG approaches, identifying trade-offs in output quality and engineering workflow alignment.
- **Selection Framework:** A metric-driven framework to guide practitioners in selecting or hybridizing LLM architectures for industrial RCA tasks.

The remainder of this paper is organized as follows: Section **??** reviews related work; Section IV details the experimental setup; Section V presents the quantitative findings; and Section VI concludes the study.

II. RESEARCH QUESTIONS

To address the challenges of automating software diagnostic reporting, this study investigates the following research questions:

- **RQ1:** To what extent can Large Language Models (LLMs) be Fine-Tuning on domain-specific fault report data to generate lexically and semantically accurate RCA explanations?
- **RQ2:** How do supervised Fine-Tuning (SFT) architectures (e.g., $T5$, $GPT-2$) compare to Retrieval-Augmented Generation (RAG) paradigms regarding output quality and alignment with engineering workflows?
- **RQ3:** How do the computational overhead and the requirement for domain-specific grounding influence the choice between SFT and RAG for industrial software maintenance?

Consistent with the technical scope of this study, model performance is evaluated exclusively through automated Natural Language Generation (NLG) metrics, specifically BLEU, ROUGE, and BERTScore. While human-centric evaluation (e.g., expert-rated utility) is recognized as a vital

component of practical deployment, it remains outside the scope of this baseline quantitative assessment.

III. RELATED WORK

1) Traditional ML Approaches in RCA: Root cause analysis (RCA) using machine learning has gained significant traction across domains such as software engineering, manufacturing, and networking. Traditional approaches often apply supervised models like support vector machines (SVMs), decision trees, or Naïve Bayes classifiers to classify faults based on historical data.

Wang et al. [5] train SVMs on textual bug reports to predict fault types (e.g., concurrency, memory, semantic bugs), achieving solid accuracy but providing no interpretability or semantic insight to support engineers during resolution. Chen et al. [6] combine PCA and SVM for fault detection in manufacturing, successfully identifying key structured features but excluding natural language inputs. Similarly, Zhang et al. [7] apply ML to SDN faults using time-series data, offering fast classification but limited diagnostic support.

Among interpretable models, Jovic et al. [8] use Naïve Bayes to identify predictive terms in bug descriptions, offering limited lexical guidance to engineers. However, none of these works integrate semantic retrieval or lexical pattern analysis to directly aid fault resolution.

2) Neural Network and Deep Learning Approaches in RCA: Deep learning has been increasingly adopted for root cause analysis (RCA), particularly in domains like manufacturing, network management, and software systems. Unlike traditional machine learning approaches, deep neural networks offer powerful feature learning capabilities, but not all methods provide semantic insight or lexical interpretability needed to support engineers in resolving issues.

Artificial neural networks (ANNs) and residual causal models like Sparse Causal Residual Neural Networks (SCRNN) have been applied to structured sensor data in manufacturing and quality control tasks [9]. These models perform well in predicting fault contributors from numerical logs but do not operate on natural language input or offer semantic interpretability.

Other approaches leverage deep learning architectures such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), autoencoders, and transformers for unsupervised anomaly detection in system logs [10]. While these methods are useful for identifying outliers, they typically lack causal reasoning and are not designed to interpret textual summaries or aid fault resolution.

More promising results are seen in graph-based models. KGroot [11], a recent framework combining knowledge graphs with graph convolutional networks (GCNs), constructs semantic fault graphs from event traces and textual logs. It enables similarity-based retrieval of past faults with known causes, providing contextual and semantic guidance. This makes it particularly well-suited for cases where fault descriptions, priorities, and summaries are available.

Another technique involves combining LSTM networks with Bayesian neural networks (BNNs) to model probabilistic causality across time-series data [12]. Though effective in capturing causal structures, it relies entirely on structured logs and does not process natural language input.

Summary of Related Work: Traditional machine learning and deep learning methods have laid the foundation for automated root cause analysis, especially in classification and clustering tasks. However, these approaches often fall short in dealing with unstructured, domain-specific text where lexical and semantic nuance is crucial. Recent work using graph-based or retrieval-augmented models has improved case similarity detection, yet lacks generalization across diverse linguistic expressions. This opens the door for large language models (LLMs), which inherently support contextual understanding, semantic retrieval, and natural language generation. As such, this thesis explores whether LLMs—specifically T5, GPT-2, and RAG—can serve not as replacements but as intelligent assistants to engineers performing RCA, addressing both relevance and clarity in fault resolution.

IV. METHODOLOGY

This section details the research methodology, experimental setup, data preparation, and evaluation procedures used to compare the performance of Large Language Models (LLMs) in generating Root Cause Analysis (RCA) explanations.

A. Research Approach and Scope

The study employs a **quantitative, experimental, and comparative** methodology rooted in a positivist paradigm. The objective is the objective, measurable, and replicable evaluation of different LLM architectures for RCA tasks. We utilize an experimental design to assess model outputs under controlled conditions using standardized, automated metrics (`BLEU`, `ROUGE`, `BERTScore`).

The research is strictly **limited to textual input features** (Summary, Description, Priority, Answer) and **automated evaluation metrics**, focusing on establishing a technical baseline for language-based RCA capabilities.

B. Data Source and Preprocessing

1) Data Source and Ethics: The dataset consists of **6,582 historical fault reports** sourced from Ericsson's internal **Jira system**. Each report includes structured metadata and unstructured text. In compliance with internal data governance and Data Protection Officer (DPO) approval, all personal identifiers and columns (e.g., author names, comments) were **removed** to ensure anonymity and data security. All experiments were conducted within a secure corporate intranet environment.

2) Feature Selection and Ground Truth: The final dataset utilized four key fields. The expert-written RCA serves as the **Ground Truth (Target Variable)**.

- **Input Features:** Summary, Description, and Priority (Importance Code).
- **Target Feature (Ground Truth):** Answer (the expert-written RCA).

The Answer field follows a standardized, multi-component structure, including the Condition, Description of fault, and Correction (See Table **??**).

Component	Function in RCA
Condition	Description of conditions that trigger the fault.
Description of the fault	Detailed technical explanation of the issue.
Description of the correction	The steps taken or planned to resolve the fault.

TABLE I: **Standardized Answer Structure for RCA (Root Cause Analysis)**

3) Data Cleaning and Wrangling: The raw data underwent several preprocessing steps:

1) **Consolidation:** Fault reports pointing to the same underlying resolution were grouped to prevent treating duplicates as independent training samples.
2) **Null Handling:** Entries lacking a value in the critical **Answer** column were removed.
3) **Partitioning:** The cleaned dataset was randomly partitioned into a **Training set (80%)** and a **Validation set (20%)**, stratified by Priority.

C. Model Architectures and Setup

Three distinct LLM architectures were selected to represent different paradigms in text generation relevant to RCA.

TABLE II: **Summary of Model Architectures**

Model	Type	Rationale for RCA
T5 (Base)	Encoder-Decoder (Seq2Seq)	Unified **Text-to-Text** transformation for structured output.
GPT-2 (Medium)	Autoregressive Decoder	Evaluates flexible, **free-form generative** capabilities.
RAG	Retrieval-Augmented	Evaluates benefit of **external grounding** via historical context.

1) Training Environment: Experiments were executed in a containerized **Kubeflow environment** using Python, the Hugging Face Transformers library, and PyTorch. The hardware setup included 1 GPU (16 GB memory) and $\sim$30 GB RAM.

2) Model Training and Hyperparameters: The T5 and GPT-2 models were fine-tuned on the Training set. Key hyperparameters included setting the Max Input/Output Sequence Length to **512 tokens**. Training was performed for up to 10 epochs using the AdamW optimizer with **early stopping** based on validation loss to prevent overfitting.

D. Evaluation Metrics and Procedure

Model performance was assessed using a combination of automated Natural Language Generation (NLG) metrics against the expert-written Ground Truth:

- **BLEU:** Measures **lexical fidelity** and precision based on n-gram overlap.
- **ROUGE:** Measures **completeness** and recall, focusing on unigram (`ROUGE-1`) and Longest Common Subsequence (`ROUGE-L`) overlap.
- **BERTScore:** Measures **semantic similarity** by computing the cosine similarity between contextual embeddings, accounting for meaning beyond exact word matches.

The **Evaluation Procedure** involved generating an RCA for every entry in the 20% Validation set and computing the three metric scores (BLEU, ROUGE, BERTScore) against the reserved Ground Truth answers.

V. RESULTS AND DISCUSSION

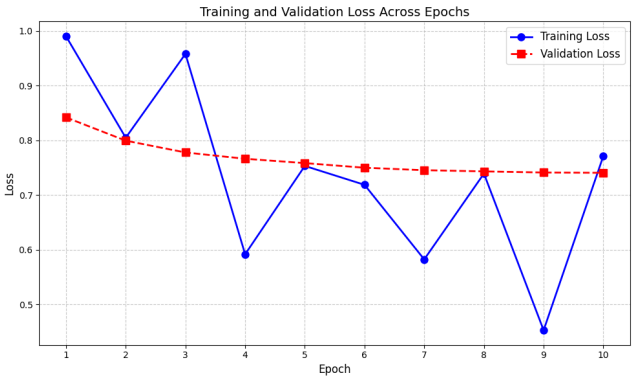
This section presents the quantitative evaluation of the three model architectures: **T5**, **GPT-2**, and **Retrieval-Augmented Generation (RAG)**. Performance is assessed on the 20% validation set using standard Natural Language Generation (NLG) metrics.

A. Training Dynamics Analysis

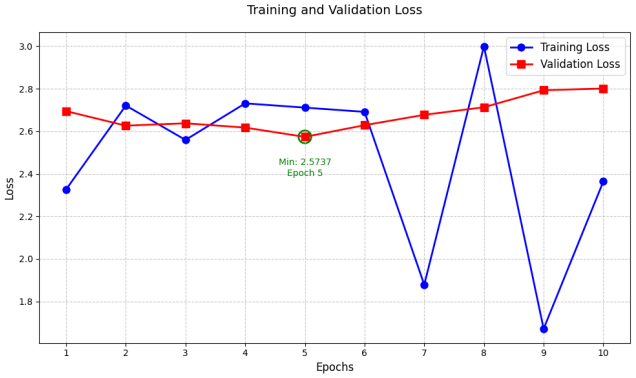
The fine-tuning process for the generative models (T5 and GPT-2) was monitored to assess stability and generalization.

- **T5 Performance:** The T5 model exhibited minor fluctuations in training loss but maintained a **stable validation loss** throughout all 10 epochs. This stability indicates that the Encoder-Decoder architecture generalized effectively to the unseen fault report data.
- **GPT-2 Performance:** The GPT-2 model showed significant **instability and erratic fluctuation** in training loss (ranging between 1.67 and 3.00). This inconsistent convergence suggests that the autoregressive, decoder-only architecture struggled to adapt to the structured, sequence-to-sequence nature of the RCA task.

The performance of the RAG model is evaluated primarily through its end-to-end output metrics in Section V-B, as generation loss alone does not fully capture its effectiveness.



(a) T5 Training & Validation Loss



(b) GPT-2 Training & Validation Loss

Fig. 1: T5 and GPT-2 Training and Validation Loss Curves

B. Quantitative Metric Comparison

The models were evaluated by comparing their generated RCA explanations against the expert-written ground truth using lexical overlap metrics (BLEU, ROUGE) and semantic similarity (BERTScore). The results are summarized in Table III.

TABLE III: Model Performance Comparison on RCA Generation

Model	BLEU-4	ROUGE-1	ROUGE-L	ROUGE-2	BERT Score
T5	**0.1810**	**0.3869**	**0.3463**	**0.2821**	0.5478
GPT-2	0.1210	0.1885	0.1001	0.0432	0.4674
RAG	0.0037	0.0682	0.0447	0.0072	**0.7715**

1) Lexical and Structural Fidelity: The **T5** model achieved the highest scores across all lexical metrics (BLEU-4, ROUGE-1, ROUGE-L, ROUGE-2). This demonstrates its ability to reproduce the specific terminology and structured format required for formal RCA documentation. The **RAG** model recorded the lowest lexical scores (BLEU-4 $= 0.0037$), reflecting its tendency to paraphrase retrieved information rather than matching exact n-grams from the ground truth.

2) Semantic Similarity: The **RAG** model achieved the highest semantic similarity score (BERTScore $= 0.7715$). Since BERTScore evaluates meaning beyond exact word matches, this finding is crucial: the RAG model generates contextually and technically equivalent explanations, highlighting its strength in knowledge grounding. The **T5** model's BERTScore of 0.5478 confirmed a strong balance between structured output and semantic correctness.

3) Metric Sensitivity and Architectural Divergence: A notable discrepancy is observed in the RAG model's performance, which recorded a near-zero BLEU-4 score (0.0037) despite achieving the highest BERTScore (0.7715). This divergence highlights the limitations of n-gram-based metrics for retrieval-based systems. While T5 is optimized to minimize cross-entropy loss against a specific ground-truth string (leading to high lexical fidelity), the RAG model synthesizes information from historical contexts that may be semantically identical but linguistically distinct from the target answer. The high BERTScore confirms that the RAG model captures the technical essence of the fault resolution, even when it does not mimic the exact phrasing of the original engineer.

C. Synthesis of Findings

The evaluation demonstrates a distinct trade-off in architectural performance for RCA:

- **T5 excels in structured reporting and automation,** achieving the highest lexical scores necessary for outputs that conform to rigid documentation standards.
- **RAG excels in semantic relevance and reasoning,** achieving the highest BERTScore which is critical for providing deep, contextually-grounded insights valuable to a human engineer.

These results provide the quantitative basis for guiding the selection of LLM architectures in practical engineering workflows.

VI. Conclusion and Future Work

This study investigated the application of Large Language Models (LLMs) for automated Root Cause Analysis (RCA) in software fault reports, comparing Fine-Tuning generative models ($T5$, $GPT-2$) with a Retrieval-Augmented Generation (RAG) approach.

A. Summary of Findings

The experimental results provide several key insights:

- **Architectural Performance:** Fine-tuning domain-specific LLMs significantly enhances diagnostic accuracy. $T5$ emerged as the superior model for structured reporting, achieving the highest lexical fidelity. Conversely, RAG demonstrated superior semantic alignment. This suggests that lexical metrics like BLEU may penalize RAG systems for generative creativity, whereas

semantic metrics like BERTScore more accurately reflect their utility in providing diverse yet technically sound diagnostic insights.

- **Metric-Driven Evaluation:** Automated metrics successfully captured complementary aspects of model performance. While *BLEU* and *ROUGE* identified the structural strengths of *T*5, *BERTScore* highlighted the interpretive value of *RAG*, establishing a systematic foundation for evaluating RCA tasks without immediate human intervention.
- **Selection Framework:** We introduced a three-dimensional framework—comprising Output Quality, Domain Alignment, and Usability—to guide model selection. The analysis indicates that a **hybrid RAG-T5 approach** is likely the most robust strategy for balancing structural documentation requirements with semantic precision.

B. Contributions and Limitations

This research establishes a replicable baseline for LLM-based RCA in software engineering. However, the study's scope was constrained by: (i) an exclusive focus on textual data, omitting multimodal diagnostic signals such as logs and charts; (ii) inherent noise in the expert-written ground truth labels; and (iii) the use of automated metrics as a proxy for human expert judgment. Despite these constraints, the study demonstrates that domain-specific fine-tuning makes LLM integration feasible for real-world fault management.

C. Future Work

Future research will build upon this foundation in the following directions:

- **Hybrid Prototype Development:** Implementing an integrated RCA assistant that combines *RAG*'s retrieval capabilities with *T*5's generation quality to support interactive engineering workflows.
- **Multimodal Integration:** Extending model architectures to process heterogeneous data sources, including system logs, performance metrics, and commit histories, to improve diagnostic depth.
- **Human-in-the-Loop Validation:** Conducting qualitative studies with domain experts to assess the interpretability, trust, and practical usability of model-generated RCA explanations.
- **Scalability and Generalization:** Evaluating larger model variants (e.g., *T*5-Large, $GPT-4$) across diverse industrial datasets to mitigate training instabilities and enhance cross-domain generalization.

References

[1] D. F. Pedroso, L. Almeida, L. E. G. Pulcinelli, W. A. A. Aisawa, I. Dutra, and S. M. Bruschi, "Anomaly detection and root cause analysis in cloud-native environments using large language models and bayesian networks," *IEEE Access*, vol. 13, pp. 77 550–77 564, 2025. DOI: 10.1109/ACCESS.2025.3565220.

[2] B. Afshinpour, M.-R. Amini, and R. Groz, "Semantic log partitioning: Towards automated root cause analysis," in *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*, 2024, pp. 639–648. DOI: 10.1109/QRS62785.2024.00069.

[3] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 674–688.

[4] Y. Han, Q. Du, Y. Huang, J. Wu, F. Tian, and C. He, "The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier," in *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024, pp. 931–943.

[5] Y. Wang et al., "Root cause prediction based on bug reports," *arXiv preprint arXiv:2103.02372*, 2021.

[6] X. Chen et al., "Big data oriented root cause identification approach based on pca and svm for product infant failure," *Journal of Intelligent Manufacturing*, 2017.

[7] H. Zhang et al., "Machine learning based root cause analysis for sdn networks," in *Proceedings of IEEE ICNC*, 2022.

[8] A. Jovic et al., "Root cause classification using naïve bayes on software defect reports," *Automatic Control and Computer Sciences*, 2023.

[9] X. Chen et al., "Big data–oriented root cause identification approach based on pca and svm for product infant failure," *Frontiers in Manufacturing Technology*, 2022.

[10] R. Singh et al., "Deep learning for automated anomaly detection in root cause analysis," *arXiv preprint arXiv:2101.07802*, 2021.

[11] L. Zhao et al., "Kgroot: Root cause analysis in microservices using knowledge graphs and graph convolutional networks," *Expert Systems with Applications*, vol. 235, p. 121 155, 2024.

[12] J. Li et al., "Root cause analysis based on lstm and bayesian neural network for time series fault data," *Algorithms*, vol. 18, no. 1, p. 11, 2023.